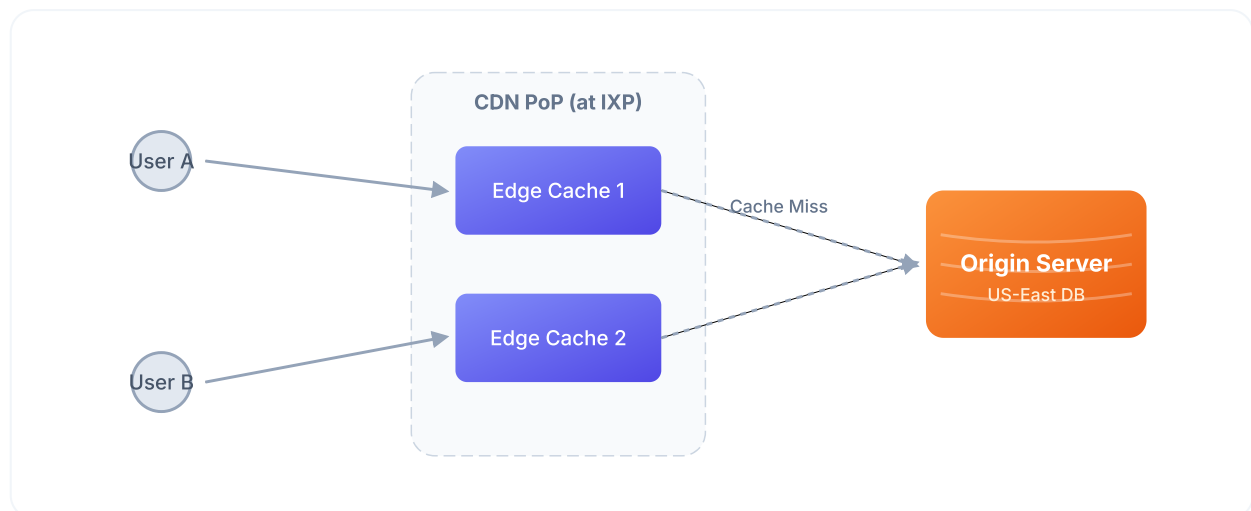


Content Delivery Networks

Beyond "I'll put a CDN here." Understand the underlying mechanics, trade-offs, and edge cases required for SDE 2 system design interviews.

1 Architecture: PoPs & Edge Servers

A CDN isn't just a random scattering of servers. They are strategically placed in **PoPs (Points of Presence)**. PoPs are highly dense data centers usually located at **IXPs (Internet Exchange Points)**. IXPs are physical locations where different Internet Service Providers (ISPs) plug their networks into each other.



Deep Dive

By placing servers exactly where the traffic crosses networks (IXPs), CDNs drastically cut down the number of network "hops" a data packet has to make, bypassing congested public internet routes.

2 The "Why": Beyond Just Latency



TLS/SSL Termination

Handshaking a secure HTTPS connection takes multiple round trips. If the user is in Mumbai and origin in NY, this takes hundreds of milliseconds. **CDNs terminate TLS at the Edge.** They handle heavy crypto math locally with the user, maintaining a persistent, optimized connection back to the origin.



Egress Cost Reduction

Cloud providers (AWS, GCP) charge heavily for data leaving their network (egress). Every byte served directly by the CDN cache is a byte you don't pay egress fees for on your expensive Origin server. This is often the primary reason companies adopt CDNs.

3 Edge Routing Mechanisms

How does a user's browser know to go to the Edge Server instead of your Origin Server? There are two primary mechanisms you must know for an SDE 2 interview.

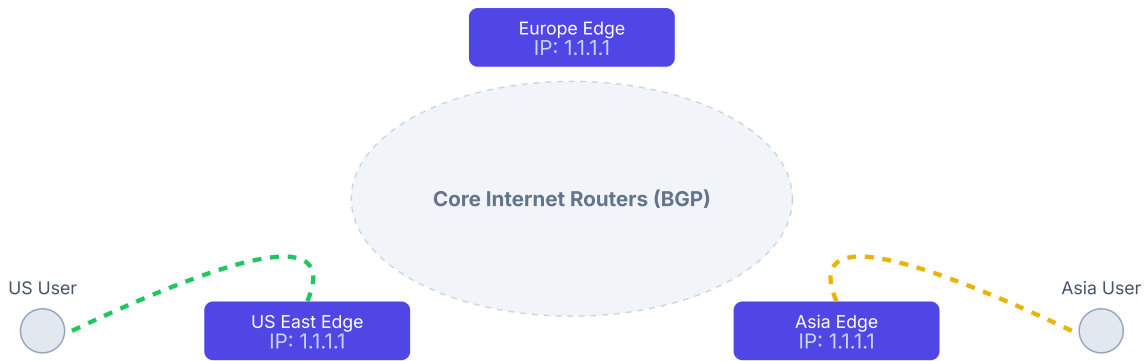
Option A: DNS Resolution (Geo-Routing)

You map your domain (`images.app.com`) via a CNAME to the CDN. The CDN's intelligent DNS server intercepts the request, looks at the user's IP (via EDNS Client Subnet), determines their geographic location, and replies with the specific IP address of the closest PoP.

User (Tokyo IP) → DNS Check → Returns Tokyo Edge IP (192.0.2.1)

Option B: Anycast Routing (BGP)

The CDN uses **one single IP address globally** (e.g., `1.1.1.1`). Through BGP (Border Gateway Protocol), multiple CDN PoPs worldwide announce they own that exact same IP. The internet's core routers automatically send packets to the topologically closest PoP.



Why Anycast is better: If a PoP crashes, BGP automatically updates routing tables, and traffic routes to the *next* closest PoP instantly. With DNS routing, you have to wait for the DNS cache (TTL) to expire before user devices recognize the redirect.

4 Caching Strategies

↓ Pull (Reactive / Read-Through)

How it works: User requests file. Edge checks cache. On Miss, Edge fetches from Origin, caches it, returns it.

When to use: Massive catalogs where users only access a small subset (e.g., Facebook images, Twitter avatars).

Trade-offs: First user takes a latency hit. Minimal setup required.

↑ Push (Proactive / Warming)

How it works: Backend API uploads data to CDN before users ask for it.

When to use: Predictable massive spikes. (e.g., Game patch day, new Netflix show release, HLS live streams).

Trade-offs: 100% cache hit rate. Higher cost (storing unrequested data). Requires pipeline logic.

Crucial Concept: Tiered Caching

If you have 200 edge nodes globally and an image expires, all 200 nodes might get a request and hit your Origin simultaneously, essentially DDOSing yourself. To prevent this, CDNs use **Tiered Caching**. Edge nodes don't hit your Origin; they hit a "Regional Mid-Tier Cache." Only if the mid-tier doesn't have it, does *one* single request go to your origin server.

5 Cache Invalidation (The Hard Part)

Data on a CDN is "Eventually Consistent." You have to engineer how it updates.

1 Time-to-Live (TTL)

Set via HTTP headers (`Cache-Control: max-age=3600`). The edge serves the file from memory for 3600 seconds. Afterward, it's marked "stale" and re-fetched on the next request.

2 Object Versioning (Cache Busting) ★ Industry Standard

You don't update `main.js`. Your pipeline creates `main.a3f9b.js`. Because the URL changes, the CDN treats it as a brand new file, fetching it instantly. No waiting for TTLs or purges.

3 Purge APIs

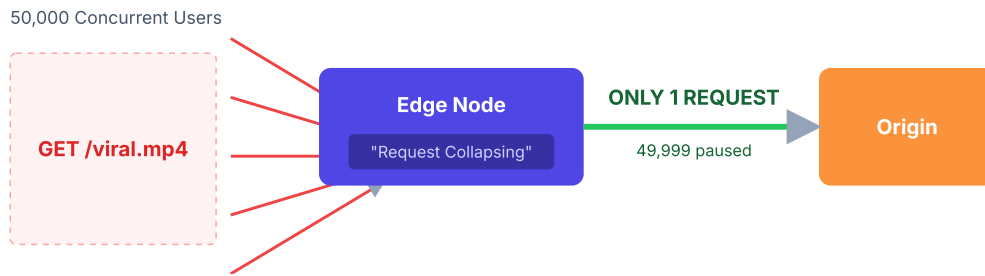
Sending an API call to the CDN: "Delete `/image.png` globally." **Warning:** This is expensive, slow, and unreliable. It can take minutes to propagate. Avoid if possible.

6 SDE 2 Interview Gotchas

⚠ The Thundering Herd (Cache Stampede)

Scenario: A highly popular video's TTL expires. At that exact millisecond, 50,000 users request it. The Edge has a cache miss for all 50k requests. It forwards all 50k to your Origin, crashing your backend instantly.

The Fix: Request Collapsing (Coalescing)



The Edge node sees 50,000 identical concurrent requests. It pauses 49,999 of them, sends *one* request to the Origin, caches the response, and then fulfills all 50,000 paused requests locally.

Edge Compute (Moving Logic to the Edge)

Modern CDNs (Cloudflare Workers, Lambda@Edge) allow running lightweight code (JS/Rust) directly on the edge server.

- **Auth Validation:** Validate a user's JWT at the Edge. If invalid, reject immediately. Saves a full round trip to backend.
- **A/B Testing:** The Edge logic modifies headers or routing based on cookies to serve different UI versions without hitting the origin.
- **Image Optimization:** Dynamically resize images or convert to WebP on the fly based on the user's device `User-Agent` string.